

دانشگاه صنعتی شریف

دانشکده مهندسی کامپیوتر

گزارش آزمایش ۹

پیاده‌سازی حافظه‌ی شرکت‌پذیر نوع سه‌گانه

Digital Systems Design Laboratory

اعضای گروه

پوریا عرفانزاده ۴۰۱۱۱۰۹۱۸
فاطیما تیمارچی ۴۰۲۱۰۵۸۰۲
آیه صابری ۴۰۲۱۰۶۱۴۵

نام درس

آزمایشگاه طراحی سیستم‌های دیجیتال

استاد درس

دکتر علیرضا اجلالی

تابستان ۱۴۰۵

۱.۹ مقدمه و اهداف

هدف این آزمایش، طراحی و پیاده‌سازی یک حافظهٔ شرکت‌پذیر[۱] نوع سه‌گانه[۲] است. مطابق دستورکار، این پیمانه باید به اندازهٔ ۱۶ ثبات ۱۶ بیتی باشد.

تفاوت بنیادی این حافظه با حافظهٔ متعارف در جهت آدرس‌دهی است. در حافظهٔ معمولی، آدرس داده می‌شود و محتوا بازگردانده می‌شود؛ در حافظهٔ شرکت‌پذیر عکس این اتّفاق می‌افتد: محتوا داده می‌شود و آدرسی که آن محتوا در آن ذخیره شده بازگردانده می‌گردد. آنچه نوع سه‌گانه را از حافظهٔ شرکت‌پذیر عادی جدا می‌کند، این است که در هر خانه افزون بر مقادیر صفر و یک، می‌توان مقدار سوم X را نیز ذخیره کرد. دستورکار معنای این مقدار را روشن کرده است: مقایسه تنها برای جایگاه‌هایی انجام می‌شود که صفر یا یک‌اند و در جایگاه X مقایسه‌ای صورت نمی‌گیرد و بیت‌ها برابر فرض می‌شوند.

دستورکار برای روشن شدن این رفتار مثالی نیز آورده است: اگر دادهٔ 01101110 در چنین حافظه‌ای جست‌وجو شود، با هر یک از الگوهای 0110XXXX، 0X1X11X0 و X1101XXX به‌عنوان انطباق[۳] در نظر گرفته می‌شود. این مثال در بخش شبیه‌سازی همین گزارش، بیت‌به‌بیت بازسازی و آزموده شده است.

روند انجام آزمایش

ابتدا شیوهٔ نمایش مقدار سه‌گانه در سخت‌افزار دودویی تعیین و رابطهٔ منطقی جست‌وجو استخراج شد. سپس پیمانهٔ TCAM به‌صورت پارامتری نوشته شد تا هم پیکربندی خواسته‌شدهٔ دستورکار را بپوشاند و هم به آن وابسته نماند. برای ارزیابی، دو میز آزمون[۴] خودآزما[۵] ساخته شد: یکی برای پیکربندی پیش‌فرض که هفت سناریو از جمله مثال خودِ دستورکار را می‌آزماید، و دیگری برای پیکربندی متفاوت که پارامتری بودن واقعی طرح را نشان می‌دهد. پس از تأیید در ModelSim، طرح در Quartus II سنتز و با یک پروندهٔ قیود طراحی تحلیل زمانی شد.

[1] Content Addressable Memory (CAM)
[2] Ternary CAM (TCAM)
[3] Match
[4] Testbench
[5] Self-Checking Testbench

۲.۹ نمایش مقدار سه‌گانه و رابطه‌ی جست‌وجو

الف) چرا هر خانه به دو ثبات نیاز دارد

سخت‌افزار دودویی نمی‌تواند سه مقدار را در یک بیت نگه دارد. بنابراین هر خانهٔ TCAM با دو ثبات هم‌عرض نمایش داده می‌شود: یک ثبات داده و یک ثبات نقاب[۶]. قرارداد به‌کاررفته در این طرح چنین است:

- بیت نقاب برابر صفر: بیت متناظر معنادار است و باید با بیت کلید جست‌وجو برابر باشد.
- بیت نقاب برابر یک: بیت متناظر بی‌اهمّیّت[۷] است و همان مقدار X دستورکار را نمایندگی می‌کند.

با این قرارداد، هر جایگاه سه‌گانه با یک زوج (نقاب,داده) کد می‌شود و مقدار بیت داده در جایگاه‌های نقاب‌شده بی‌اثر است.

بیت نقاب	بیت داده	مقدار ذخیره‌شده	رفتار در مقایسه
0	0	0	باید با بیت کلید برابر باشد
0	1	1	باید با بیت کلید برابر باشد
1	–	X	هر مقداری در کلید پذیرفته می‌شود

جدول ۱: کدگذاری مقدار سه‌گانه با زوج داده و نقاب

ب) رابطه‌ی جست‌وجو

با این کدگذاری، شرط انطباق یک خانه با کلید جست‌وجو به یک رابطهٔ بیتی ساده تبدیل می‌شود. برای خانهٔ iام:

$$\text{انطباق}_i \;=\; \Big((\text{داده}_i \oplus \text{کلید}) \;\wedge\; \overline{\text{نقاب}_i} \Big) = 0.$$

منطق این رابطه گام‌به‌گام چنین است. یای‌انحصاری میان کلید و داده، در هر جایگاهی که این دو با هم فرق دارند مقدار یک می‌دهد؛ یعنی نقشهٔ اختلاف‌ها را می‌سازد. واو کردن این نقشه با نقیض نقاب، اختلاف‌های واقع در جایگاه‌های بی‌اهمّیّت را حذف می‌کند و تنها اختلاف‌های واقع در جایگاه‌های معنادار را باقی می‌گذارد. سرانجام، صفر بودن کلّ نتیجه یعنی هیچ اختلاف معناداری وجود ندارد و خانه با کلید منطبق است.

[6] Mask
[7] Don't-Care

برای نمونه، مثال خودِ دستورکار را با این رابطه بررسی می‌کنیم. کلید 01101110 و الگوی 0110XXXX را در نظر بگیرید که با داده 01100000 و نقاب 00001111 کد می‌شود:

$$\text{کلید} \oplus \text{داده} = 01101110 \oplus 01100000 = 00001110,$$

$$\overline{\text{نقاب}} = 11110000,$$

$$00001110 \wedge 11110000 = 00000000 \;\Rightarrow\; \text{انطباق}.$$

اختلاف‌ها همگی در چهار جایگاه پایین رخ داده‌اند که دقیقاً همان جایگاه‌های نقاب‌شده‌اند، پس نادیده گرفته می‌شوند.

ج) بیت اعتبار

یک نکتهٔ ظریف در همین رابطه نهفته است: خانه‌ای که هرگز نوشته نشده و محتوای آن صفر است، در برابر کلید صفر انطباق نشان می‌دهد. بدتر از آن، خانه‌ای با نقاب سراسر یک با هر کلیدی منطبق می‌شود. برای جلوگیری از این انطباق‌های کاذب، به ازای هر خانه یک بیت اعتبار[8] نگه داشته می‌شود که تنها با نوشتن در آن خانه یک می‌شود و با بازنشانی صفر می‌گردد. شرط نهایی انطباق، واو منطقی این بیت با رابطهٔ بالاست.

۳.۹ توصیف پیمانه

الف) واسط و پارامترها

پیمانه با سه پارامتر نوشته شده است: پهنای داده، عمق حافظه، و پهنای آدرس که به‌طور خودکار از عمق محاسبه می‌شود. مقادیر پیش‌فرض همان ۱۶ و ۱۶ خواسته‌شدهٔ دستورکار است.

[8] Valid Bit

نام	جهت	پهنا	نقش
clk	ورودی	۱	ساعت مدار
rstN	ورودی	۱	بازنشانی ناهم‌زمان فعّال‌صفر
r_e	ورودی	۱	اجازه‌ی جست‌وجو
w_e	ورودی	۱	اجازه‌ی نوشتن؛ بر جست‌وجو اولویّت دارد
data_in	ورودی	۱۶	دادهٔ نوشتنی یا کلید جست‌وجو
mask	ورودی	۱۶	نقاب خانه در هنگام نوشتن
addr_in	ورودی	۴	آدرس خانه در هنگام نوشتن
matched_num	خروجی	۱۶	دادهٔ ذخیره‌شده‌ی خانه‌ی منطبق
matched_addr	خروجی	۴	آدرس خانه‌ی منطبق
match_vector	خروجی	۱۶	یک بیت به ازای هر خانه؛ همه‌ی انطباق‌ها
match	خروجی	۱	دست‌کم یک انطباق وجود دارد

جدول ۲: واسط پیمانه‌ی TCAM

دربارهٔ نام matched_num باید دقّت کرد: این خروجی شمار انطباق‌ها نیست، بلکه *دادهٔ ذخیره‌شدهٔ* خانه‌ای است که منطبق شده است. شمار و مکان انطباق‌ها را باید از match_vector خواند.

ب) جست‌وجوی موازی و اولویّت آدرس

جست‌وجو در یک بلوک ترکیبی انجام می‌شود که با یک حلقهٔ for روی همهٔ خانه‌ها می‌گردد. این حلقه در زمان سنتز باز می‌شود و به شانزده مقایسه‌کنندهٔ مستقل تبدیل می‌گردد که همگی هم‌زمان کار می‌کنند؛ یعنی جست‌وجو ذاتاً موازی است و زمان آن به شمار خانه‌ها بستگی ندارد. همین ویژگی، دلیل اصلی استفاده از این نوع حافظه در کاربردهایی مانند جدول مسیریابی است.

وقتی بیش از یک خانه منطبق شود، باید تصمیمی دربارهٔ خروجی تک‌مقداری گرفت. در این طرح، چون حلقه از آدرس کوچک به بزرگ پیش می‌رود و هر انطباق تازه مقدار پیشین را بازنویسی می‌کند، آدرس *بزرگ‌ترین* خانهٔ منطبق در خروجی می‌ماند. در مقابل، match_vector هیچ اطّلاعاتی را دور نمی‌ریزد و همهٔ انطباق‌ها را گزارش می‌کند. این تفکیک عمدی است: خروجی تک‌مقداری برای کاربرد متعارف و بردار برای زمانی که شمار یا مجموعهٔ انطباق‌ها اهمّیّت دارد.

ج) نوشتن، جست‌وجو و اولویّت میان آن‌ها

بلوک هم‌زمان طرح سه شاخه دارد. بازنشانی ناهم‌زمان[9] همهٔ بیت‌های اعتبار و خروجی‌ها را صفر می‌کند. اگر اجازهٔ نوشتن فعّال باشد، داده و نقاب در خانهٔ آدرس‌داده‌شده نوشته و بیت اعتبار آن یک می‌شود. تنها در صورتی که نوشتن فعّال نباشد و اجازهٔ جست‌وجو فعّال باشد، نتیجهٔ بلوک ترکیبی در ثبات‌های خروجی ثبت می‌گردد.

ترتیب این شاخه‌ها یعنی نوشتن بر جست‌وجو اولویّت دارد. این انتخاب از ابهام جلوگیری می‌کند: اگر هر دو اجازه هم‌زمان فعّال شوند، روشن است که در آن پالس ساعت نوشتن انجام می‌شود و خروجی‌های جست‌وجو دست‌نخورده می‌مانند. همچنین توجّه شود که خروجی‌ها ثبت‌شده‌اند و نه ترکیبی؛ یعنی نتیجهٔ جست‌وجو یک پالس ساعت پس از اعمال کلید در خروجی ظاهر می‌شود و تا جست‌وجوی بعدی پایدار می‌ماند.

وارسی $\text{addr_in} < \text{DEPTH}$ نیز عمدی است و تنها هنگامی اهمّیّت پیدا می‌کند که عمق حافظه توانی از دو نباشد؛ در چنین حالتی، پهنای آدرس بیش از عمق واقعی مقدار می‌پذیرد و بدون این وارسی، نوشتن در آدرس خارج از محدوده رفتار نامعیّن می‌ساخت.

```
// ================================================================
// Experiment 9 - Parameterized TCAM
// Default configuration: 16 entries x 16 bits
//
// mask bit = 1  -> stored bit is Don't-Care (X)
// mask bit = 0  -> stored bit must equal the search bit
//
// Search equation for each valid entry:
// ((search_key XOR stored_data) AND NOT stored_mask) == 0
// ================================================================

module TCAM #(
    parameter integer DATA_WIDTH = 16,
    parameter integer DEPTH      = 16,
    parameter integer ADDR_WIDTH = (DEPTH <= 1) ? 1 : $clog2(DEPTH)
)(
    input                           clk,
    input                           rstN,       // asynchronous active-low reset
    input                           r_e,        // search/read enable
    input                           w_e,        // write enable; has priority over r_e
    input      [DATA_WIDTH-1:0]     data_in,    // write data or search key
    input      [DATA_WIDTH-1:0]     mask,       // valid on write: 1 means X/Don't-Care
    input      [ADDR_WIDTH-1:0]     addr_in,    // write address

    output reg [DATA_WIDTH-1:0]     matched_num,
    output reg [ADDR_WIDTH-1:0]     matched_addr,
    output reg [DEPTH-1:0]          match_vector,
    output reg                      match
);

    // Each TCAM entry is represented by data bits plus mask bits.
    reg [DATA_WIDTH-1:0] data_mem [0:DEPTH-1];
    reg [DATA_WIDTH-1:0] mask_mem [0:DEPTH-1];
    reg [DEPTH-1:0]      valid_mem;

    // Combinational search results. They are registered when r_e = 1.
```

[9] Asynchronous Reset

```
    reg [DATA_WIDTH-1:0] search_matched_num;
    reg [ADDR_WIDTH-1:0] search_matched_addr;
    reg [DEPTH-1:0]      search_match_vector;
    reg                  search_match;

    integer i;
    integer j;

    // ------------------------------------------------------------
    // Parallel TCAM search.
    // All valid entries are compared with data_in simultaneously.
    // If multiple entries match, matched_num/matched_addr report the
    // highest matching address. match_vector reports every match.
    // ------------------------------------------------------------
    always @(*) begin
        search_matched_num  = {DATA_WIDTH{1'b0}};
        search_matched_addr = {ADDR_WIDTH{1'b0}};
        search_match_vector = {DEPTH{1'b0}};
        search_match        = 1'b0;

        for (i = 0; i < DEPTH; i = i + 1) begin
            if (valid_mem[i] &&
                (((data_in ^ data_mem[i]) & (~mask_mem[i])) == {DATA_WIDTH{1'b0}})) begin
                search_match_vector[i] = 1'b1;
                search_match           = 1'b1;
                search_matched_num     = data_mem[i];
                search_matched_addr    = i[ADDR_WIDTH-1:0];
            end
        end
    end

    // ------------------------------------------------------------
    // Storage and output registers.
    // Write has priority over search if w_e and r_e are both asserted.
    // ------------------------------------------------------------
    always @(posedge clk or negedge rstN) begin
        if (!rstN) begin
            matched_num  <= {DATA_WIDTH{1'b0}};
            matched_addr <= {ADDR_WIDTH{1'b0}};
            match_vector <= {DEPTH{1'b0}};
            match        <= 1'b0;
            valid_mem    <= {DEPTH{1'b0}};

            // valid_mem alone is enough to invalidate entries, but clearing
            // the arrays also keeps simulation state deterministic.
            for (j = 0; j < DEPTH; j = j + 1) begin
                data_mem[j] <= {DATA_WIDTH{1'b0}};
                mask_mem[j] <= {DATA_WIDTH{1'b0}};
            end
        end
        else if (w_e) begin
            // This check matters when DEPTH is not an exact power of two.
            if (addr_in < DEPTH) begin
                data_mem[addr_in]  <= data_in;
                mask_mem[addr_in]  <= mask;
                valid_mem[addr_in] <= 1'b1;
            end
        end
        else if (r_e) begin
            matched_num  <= search_matched_num;
            matched_addr <= search_matched_addr;
            match_vector <= search_match_vector;
            match        <= search_match;
        end
    end

endmodule
```

کد ۱: پیمانه‌ی TCAM پارامتری

۴.۹ میزهای آزمون

دو میز آزمون خودآزما نوشته شد. میز آزمون نخست پیکربندی خواسته‌شدهٔ دستورکار را با هفت سناریو می‌آزماید و میز آزمون دوم، همان پیمانه را با پیکربندی متفاوتی اجرا می‌کند.

سناریو	عنوان	آنچه سنجیده می‌شود
1	خانه‌های نامعتبر	پس از بازنشانی نباید هیچ انطباق کاذبی رخ دهد
2	نقاب‌های متعارف	سه خانه با نقاب‌های گوناگون و سه کلید
3	انطباق دقیق	نقاب صفر؛ تنها کلید یکسان باید منطبق شود
4	مثال دستورکار	سه الگوی Xدار در برابر کلید 01101110
5	خانه‌ی سراسر X	باید با هر کلیدی منطبق شود
6	انطباق چندگانه	بزرگ‌ترین آدرس در خروجی، همه در بردار
7	بازنویسی خانه	محتوای تازه باید جای محتوای پیشین را بگیرد

جدول ۳: هفت سناریوی میز آزمون پیکربندی پیش‌فرض

```
`timescale 1ns/1ps

// ================================================================
// Experiment 9 - Self-checking testbench for the default TCAM
// Configuration: 16 entries x 16 bits
// ================================================================
module TB;
    localparam integer DATA_WIDTH = 16;
    localparam integer DEPTH      = 16;
    localparam integer ADDR_WIDTH = (DEPTH <= 1) ? 1 : $clog2(DEPTH);

    reg                      clk;
    reg                      rstN;
    reg                      r_e;
    reg                      w_e;
    reg  [DATA_WIDTH-1:0]    data_in;
    reg  [DATA_WIDTH-1:0]    mask;
    reg  [ADDR_WIDTH-1:0]    addr_in;

    wire [DATA_WIDTH-1:0]    matched_num;
    wire [ADDR_WIDTH-1:0]    matched_addr;
    wire [DEPTH-1:0]         match_vector;
    wire                     match;

    integer errors;

    TCAM #(
        .DATA_WIDTH(DATA_WIDTH),
        .DEPTH(DEPTH)
    ) dut (
        .clk(clk),
        .rstN(rstN),
        .r_e(r_e),
        .w_e(w_e),
        .data_in(data_in),
        .mask(mask),
        .addr_in(addr_in),
        .matched_num(matched_num),
        .matched_addr(matched_addr),
        .match_vector(match_vector),
        .match(match)
    );
```

```
43
44      initial clk = 1'b0;
45      always #5 clk = ~clk;
46
47      task write_entry;
48          input [ADDR_WIDTH-1:0] a;
49          input [DATA_WIDTH-1:0] d;
50          input [DATA_WIDTH-1:0] m;
51          begin
52              @(negedge clk);
53              addr_in = a;
54              data_in = d;
55              mask    = m;
56              r_e     = 1'b0;
57              w_e     = 1'b1;
58              @(posedge clk);
59              #1;
60              w_e     = 1'b0;
61          end
62      endtask
63
64      task search_key;
65          input [DATA_WIDTH-1:0] key;
66          begin
67              @(negedge clk);
68              data_in = key;
69              r_e     = 1'b1;
70              w_e     = 1'b0;
71              @(posedge clk);
72              #1;
73              r_e     = 1'b0;
74          end
75      endtask
76
77      task expect_hit;
78          input [DATA_WIDTH-1:0] expected_data;
79          input [ADDR_WIDTH-1:0] expected_addr;
80          input [DEPTH-1:0]      expected_vector;
81          begin
82              if (match !== 1'b1 ||
83                  matched_num !== expected_data ||
84                  matched_addr !== expected_addr ||
85                  match_vector !== expected_vector) begin
86                  $display("[FAIL] expected hit: data=%h addr=%0d vector=%b",
87                           expected_data, expected_addr, expected_vector);
88                  $display("       actual      : match=%b data=%h addr=%0d vector=%b",
89                           match, matched_num, matched_addr, match_vector);
90                  errors = errors + 1;
91              end
92              else begin
93                  $display("[PASS] hit data=%h addr=%0d vector=%b",
94                           matched_num, matched_addr, match_vector);
95              end
96          end
97      endtask
98
99      task expect_miss;
100         begin
101             if (match !== 1'b0 || match_vector !== {DEPTH{1'b0}}) begin
102                 $display("[FAIL] expected miss, got match=%b data=%h addr=%0d vector=%b",
103                          match, matched_num, matched_addr, match_vector);
104                 errors = errors + 1;
105             end
106             else begin
107                 $display("[PASS] miss as expected");
108             end
109         end
110     endtask
111
112     initial begin
113         errors  = 0;
114         rstN    = 1'b0;
```

```
        r_e     = 1'b0;
        w_e     = 1'b0;
        data_in = {DATA_WIDTH{1'b0}};
        mask    = {DATA_WIDTH{1'b0}};
        addr_in = {ADDR_WIDTH{1'b0}};

        // ------------------------------------------------------
        // Test 1: reset / invalid entries must not cause false hits.
        // ------------------------------------------------------
        #12;
        rstN = 1'b1;
        search_key(16'h0000);
        expect_miss;

        // ------------------------------------------------------
        // Test 2: ordinary masked entries.
        // ------------------------------------------------------
        // addr 0: compare upper nibble and bits [7:4]; other bits are X.
        write_entry(4'd0, 16'hABCD, 16'h0F0F);

        // addr 1: upper byte is X; low byte must equal EF.
        write_entry(4'd1, 16'hCDEF, 16'hFF00);

        // addr 2: arbitrary bit-level mask.
        write_entry(4'd2, 16'h50C9, 16'hC32C);

        search_key(16'hAFC2);
        expect_hit(16'hABCD, 4'd0, 16'h0001);

        search_key(16'hD3C9);
        expect_hit(16'h50C9, 4'd2, 16'h0004);

        search_key(16'hCDFE);
        expect_miss;

        // ------------------------------------------------------
        // Test 3: exact match, mask = 0.
        // ------------------------------------------------------
        write_entry(4'd3, 16'h1234, 16'h0000);

        search_key(16'h1234);
        expect_hit(16'h1234, 4'd3, 16'h0008);

        search_key(16'h1235);
        expect_miss;

        // ------------------------------------------------------
        // Test 4: examples explicitly shoum in the experiment sheet.
        // Search word: 01101110 = 8'h6E
        // Patterns: 0110XXXX, X1101XXX, 0X1X11X0
        // The upper byte is kept exact at 00 because this TCAM is 16-bit.
        // ------------------------------------------------------

        // addr 6: 0110XXXX
        // data = 01100000, mask = 00001111
        write_entry(4'd6, 16'h0060, 16'h000F);
        search_key(16'h006E);
        expect_hit(16'h0060, 4'd6, 16'h0040);

        // addr 7: X1101XXX
        // data = 01101000, mask = 10000111
        write_entry(4'd7, 16'h0068, 16'h0087);
        search_key(16'h006E);
        expect_hit(16'h0068, 4'd7, 16'h00C0);

        // addr 8: 0X1X11X0
        // data = 00101100, mask = 01010010
        write_entry(4'd8, 16'h002C, 16'h0052);
        search_key(16'h006E);
        expect_hit(16'h002C, 4'd8, 16'h01C0);

        // ------------------------------------------------------
```

```
        // Test 5: all-X entry matches every key.
        // ------------------------------------------------------
        write_entry(4'd4, 16'h0000, 16'hFFFF);
        search_key(16'hBEEE);
        expect_hit(16'h0000, 4'd4, 16'h0010);

        // ------------------------------------------------------
        // Test 6: multiple matches. Highest address must be selected,
        // while match_vector must report all matching entries.
        // ------------------------------------------------------
        write_entry(4'd5, 16'hFFFF, 16'hFFFF);
        search_key(16'hCAFE);
        expect_hit(16'hFFFF, 4'd5, 16'h0030);

        // ------------------------------------------------------
        // Test 7: overwrite an existing entry.
        // Entry 4 is still all-X, and entry 5 becomes exact CAFE.
        // Both match CAFE; highest address 5 is returned.
        // ------------------------------------------------------
        write_entry(4'd5, 16'hCAFE, 16'h0000);
        search_key(16'hCAFE);
        expect_hit(16'hCAFE, 4'd5, 16'h0030);

        if (errors == 0)
            $display("\n===================================");
        if (errors == 0)
            $display("ALL DEFAULT-CONFIGURATION TESTS PASSED");
        else
            $display("TESTS FINISHED WITH %0d ERROR(S)", errors);
        if (errors == 0)
            $display("===================================\n");

        #10;
        $finish;
    end
endmodule
```

کد ۲: میز آزمون پیکربندی پیش‌فرض

میز آزمون دوم، پیمانه را با پهنای داده هشت بیت و عمق پنج خانه اجرا می‌کند. انتخاب عدد پنج عمدی است، چون توانی از دو نیست؛ در این حالت پهنای آدرس سه بیت می‌شود و می‌توان آدرس‌های شش و هفت را نیز روی گذرگاه گذاشت، حال آنکه خانه‌ای برای آن‌ها وجود ندارد. آخرین وارسی این میز آزمون دقیقاً همین حالت را می‌سنجد و نشان می‌دهد که نوشتن در آدرس خارج از محدوده به‌درستی نادیده گرفته می‌شود.

```
`timescale 1ns/1ps

// ================================================================
// Extra parameterization test.
// Uses a non-default configuration to prove the module is generic:
// DATA_WIDTH = 8, DEPTH = 5, automatically calculated ADDR_WIDTH = 3.
// ================================================================
module TB_param;
    localparam integer DATA_WIDTH = 8;
    localparam integer DEPTH      = 5;
    localparam integer ADDR_WIDTH = (DEPTH <= 1) ? 1 : $clog2(DEPTH);

    reg                      clk;
    reg                      rstN;
    reg                      r_e;
    reg                      w_e;
    reg  [DATA_WIDTH-1:0]    data_in;
    reg  [DATA_WIDTH-1:0]    mask;
```

```
    reg  [ADDR_WIDTH-1:0]     addr_in;

    wire [DATA_WIDTH-1:0]     matched_num;
    wire [ADDR_WIDTH-1:0]     matched_addr;
    wire [DEPTH-1:0]          match_vector;
    wire                      match;

    integer errors;

    TCAM #(
        .DATA_WIDTH(DATA_WIDTH),
        .DEPTH(DEPTH)
    ) dut (
        .clk(clk),
        .rstN(rstN),
        .r_e(r_e),
        .w_e(w_e),
        .data_in(data_in),
        .mask(mask),
        .addr_in(addr_in),
        .matched_num(matched_num),
        .matched_addr(matched_addr),
        .match_vector(match_vector),
        .match(match)
    );

    initial clk = 1'b0;
    always #5 clk = ~clk;

    task write_entry;
        input [ADDR_WIDTH-1:0] a;
        input [DATA_WIDTH-1:0] d;
        input [DATA_WIDTH-1:0] m;
        begin
            @(negedge clk);
            addr_in = a;
            data_in = d;
            mask    = m;
            r_e     = 1'b0;
            w_e     = 1'b1;
            @(posedge clk);
            #1;
            w_e = 1'b0;
        end
    endtask

    task search_key;
        input [DATA_WIDTH-1:0] key;
        begin
            @(negedge clk);
            data_in = key;
            r_e     = 1'b1;
            w_e     = 1'b0;
            @(posedge clk);
            #1;
            r_e = 1'b0;
        end
    endtask

    initial begin
        errors  = 0;
        rstN    = 1'b0;
        r_e     = 1'b0;
        w_e     = 1'b0;
        data_in = 8'h00;
        mask    = 8'h00;
        addr_in = {ADDR_WIDTH{1'b0}};

        #12;
        rstN = 1'b1;

        // Write to the highest legal address of DEPTH=5.
```

```
        // Pattern: 1010XXXX
        write_entry(3'd4, 8'b10100000, 8'b00001111);

        search_key(8'b10101111);
        if (match !== 1'b1 || matched_addr !== 3'd4 || match_vector !== 5'b10000) begin
            $display("[FAIL] parameterized masked-match test");
            errors = errors + 1;
        end
        else
            $display("[PASS] parameterized masked-match test");

        search_key(8'b10110000);
        if (match !== 1'b0 || match_vector !== 5'b00000) begin
            $display("[FAIL] parameterized miss test");
            errors = errors + 1;
        end
        else
            $display("[PASS] parameterized miss test");

        // Exact entry at another address.
        write_entry(3'd1, 8'h5A, 8'h00);
        search_key(8'h5A);
        if (match !== 1'b1 || matched_addr !== 3'd1 || matched_num !== 8'h5A || match_vector !==
    5'b00010) begin
            $display("[FAIL] parameterized exact-match test");
            errors = errors + 1;
        end
        else
            $display("[PASS] parameterized exact-match test");

        // Illegal address for DEPTH=5 must be ignored safely.
        write_entry(3'd7, 8'h3C, 8'h00);
        search_key(8'h3C);
        if (match !== 1'b0 || match_vector !== 5'b00000) begin
            $display("[FAIL] out-of-range write-address test");
            errors = errors + 1;
        end
        else
            $display("[PASS] out-of-range write-address test");

        if (errors == 0)
            $display("\nALL PARAMETERIZATION TESTS PASSED.\n");
        else
            $display("\nPARAMETERIZATION TEST FINISHED WITH %0d ERROR(S).\n", errors);

        #10;
        $finish;
    end
endmodule
```

کد ۳: میز آزمون پیکربندی پارامتری

۵.۹ اجرا در ModelSim

اجرای هر دو میز آزمون با فرمان‌های زیر انجام می‌شود.

```
vlog TCAM.v TB.v
vsim work.TB
add wave -r sim:/TB/*
add wave -r sim:/TB/dut/*
run -all
wave zoom full
```

کد ۴: فرمان‌های اجرای میز آزمون پیش‌فرض

```
vlog TCAM.v TB_param.v
vsim work.TB_param
add wave -r sim:/TB_param/*
add wave -r sim:/TB_param/dut/*
run -all
wave zoom full
```

کد ۵: فرمان‌های اجرای میز آزمون پارامتری

```
ModelSim> vlog TCAM.v TB.v
# Model Technology ModelSim ALTERA vlog 10.1d Compiler 2012.11 Nov  2 2012
# -- Compiling module TCAM
# -- Compiling module TB
#
# Top level modules:
#       TB
ModelSim> vsim work.TB
# vsim work.TB
# Loading work.TB
# Loading work.TCAM
VSIM 3> add wave -r sim:/TB/*
VSIM 4> add wave -r sim:/TB/dut/*
```

شکل ۱: آغاز اجرا و همگردانی پیمانه و میز آزمون

```
VSIM 5> run -all
# [PASS] miss as expected
# [PASS] hit data=abcd addr=0 vector=0000000000000001
# [PASS] hit data=50c9 addr=2 vector=0000000000000100
# [PASS] miss as expected
# [PASS] hit data=1234 addr=3 vector=0000000000001000
# [PASS] miss as expected
# [PASS] hit data=0060 addr=6 vector=0000000001000000
# [PASS] hit data=0068 addr=7 vector=0000000011000000
# [PASS] hit data=002c addr=8 vector=0000000111000000
# [PASS] hit data=0000 addr=4 vector=0000000000010000
# [PASS] hit data=ffff addr=5 vector=0000000000110000
# [PASS] hit data=cafe addr=5 vector=0000000000110000
#
# ====================================
# ALL DEFAULT-CONFIGURATION TESTS PASSED
# ====================================
#
# ** Note: $finish    : TB.v(220)
#    Time: 246 ns  Iteration: 0  Instance: /TB
# 1
# Break in Module TB at TB.v line 220
VSIM 6> wave zoom full
# (0 ps) (258300 ps)

VSIM 7>
```

شکل ۲: خروجی کامل میز آزمون پیش‌فرض و گزارش قبولی دوازده وارسی

هر دوازده وارسی میز آزمون نخست با موفّقیّت گذشتند و شبیه‌سازی در لحظهٔ ۲۴۶ نانوثانیه پایان یافت.

```
VSIM 7> vlog TCAM.v TB_param.v
# Model Technology ModelSim ALTERA vlog 10.1d Compiler 2012.11 Nov  2 2012
# -- Compiling module TCAM
# -- Compiling module TB_param
#
# Top level modules:
#       TB_param
VSIM 8> vsim work.TB_param
# vsim work.TB_param
# Loading work.TB_param
# Loading work.TCAM
VSIM 9> add wave -r sim:/TB_param/*
VSIM 10> add wave -r sim:/TB_param/dut/*
VSIM 11> run -all
# [PASS] parameterized masked-match test
# [PASS] parameterized miss test
# [PASS] parameterized exact-match test
# [PASS] out-of-range write-address test
#
# ALL PARAMETERIZATION TESTS PASSED.
#
# ** Note: $finish    : TB_param.v(136)
#    Time: 96 ns  Iteration: 0  Instance: /TB_param
# 1
# Break in Module TB_param at TB_param.v line 136
VSIM 12> wave zoom full
# (0 ps) (100800 ps)

VSIM 13>
```

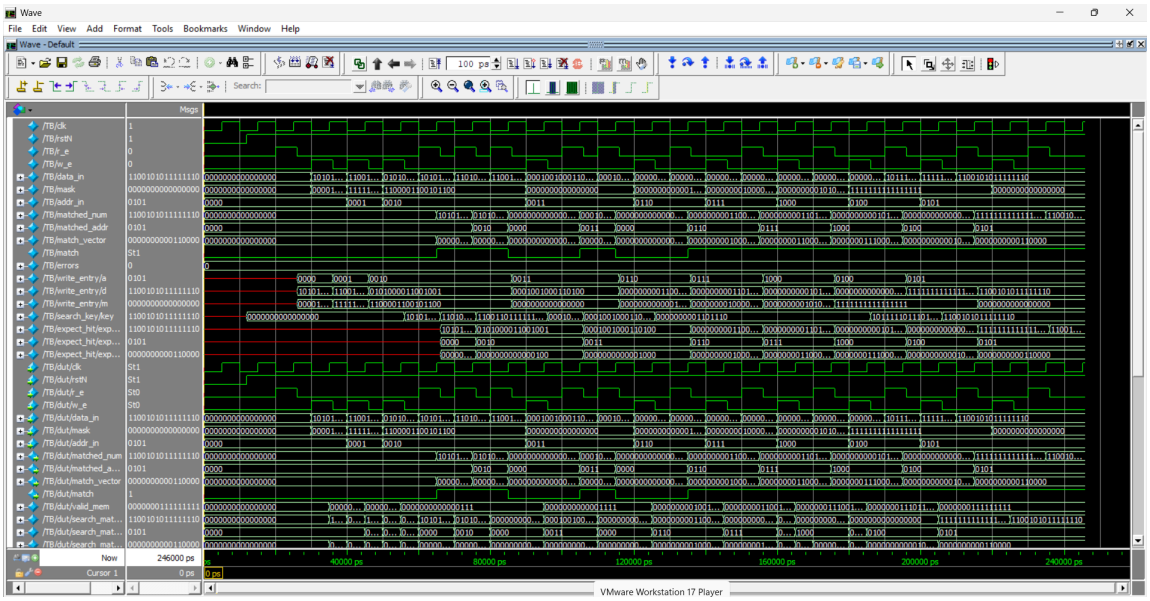
شکل ۳: خروجی میز آزمون پارامتری و گزارش قبولی چهار وارسی

۶.۹ تحلیل شکل موج‌ها

الف) کلید خواندن شکل موج

ساعت میز آزمون دورهٔ ۱۰ نانوثانیه دارد و هر عمل نوشتن یا جست‌وجو دقیقاً یک پالس ساعت طول می‌کشد. هر دو وظیفهٔ میز آزمون، ورودی‌ها را در لبهٔ پایین‌رونده اعمال می‌کنند و پیمانه در لبهٔ بالارونده بعدی آن‌ها را می‌پذیرد. بنابراین کلّ اجرا یک زنجیرهٔ منظّم از رویدادهای ده‌نانوثانیه‌ای است که از لحظهٔ ۲۵ نانوثانیه آغاز می‌شود.

یک نکته در خواندن خروجی‌ها اهمّیّت دارد: چون خروجی‌ها ثبت‌شده‌اند، نتیجهٔ هر جست‌وجو در همان لبه‌ای ظاهر می‌شود که کلید در آن نمونه‌برداری شده است و سپس تا جست‌وجوی بعدی پایدار می‌ماند. پس در شکل موج، پلّه‌های match_vector دقیقاً بر لبه‌های جست‌وجو منطبق‌اند و نه یک پالس دیرتر.



شکل ۴: شکل موج کامل اجرای میز آزمون پیش‌فرض

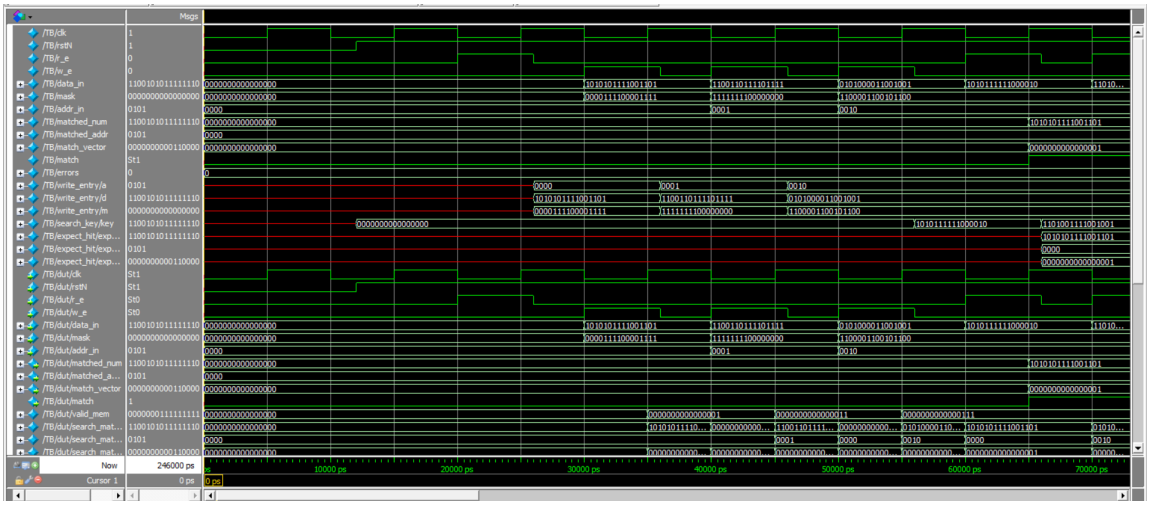
ب) دنباله‌ی کامل رویدادها

جدول زیر کلّ اجرا را رویدادبه‌رویداد دنبال می‌کند. هر سطر یک لبهٔ فعّال ساعت است.

زمان (ns)	عمل	مقادیر	نتیجه
25	جست‌وجو	کلید 0000	بدون انطباق
35	نوشتن	خانهٔ 0: داده $ABCD$، نقاب $0F0F$	—
45	نوشتن	خانهٔ 1: داده $CDEF$، نقاب $FF00$	—
55	نوشتن	خانهٔ 2: داده $50C9$، نقاب $C32C$	—
65	جست‌وجو	کلید $AFC2$	انطباق در خانهٔ 0
75	جست‌وجو	کلید $D3C9$	انطباق در خانهٔ 2
85	جست‌وجو	کلید $CDFE$	بدون انطباق
95	نوشتن	خانهٔ 3: داده 1234، نقاب 0000	—
105	جست‌وجو	کلید 1234	انطباق در خانهٔ 3
115	جست‌وجو	کلید 1235	بدون انطباق
125	نوشتن	خانهٔ 6: داده 0060، نقاب $000F$	—
135	جست‌وجو	کلید $006E$	انطباق در خانهٔ 6
145	نوشتن	خانهٔ 7: داده 0068، نقاب 0087	—
155	جست‌وجو	کلید $006E$	انطباق در خانه‌های $6, 7$
165	نوشتن	خانهٔ 8: داده $002C$، نقاب 0052	—
175	جست‌وجو	کلید $006E$	انطباق در خانه‌های $6, 7, 8$
185	نوشتن	خانهٔ 4: داده 0000، نقاب $FFFF$	—
195	جست‌وجو	کلید $BEEE$	انطباق در خانهٔ 4
205	نوشتن	خانهٔ 5: داده $FFFF$، نقاب $FFFF$	—
215	جست‌وجو	کلید $CAFE$	انطباق در خانه‌های $4, 5$
225	نوشتن	خانهٔ 5: داده $CAFE$، نقاب 0000	—
235	جست‌وجو	کلید $CAFE$	انطباق در خانه‌های $4, 5$

جدول ۴: دنباله‌ی کامل رویدادهای شبیه‌سازی؛ هر سطر یک لبه‌ی فعّال ساعت است

ج) بازه‌ی نخست: نقاب‌های متعارف



شکل ۵: نمای بزرگ‌شده‌ی بازه‌ی نخست؛ خانه‌های نامعتبر و نقاب‌های متعارف

در لحظهٔ ۲۵ نانوثانیه، پیش از آنکه چیزی نوشته شود، کلید 0000 جست‌وجو می‌شود. اگرچه محتوای همهٔ خانه‌ها در این لحظه صفر است و از نظر رابطهٔ بیتی با کلید صفر می‌خواند، خروجی match پایین می‌ماند. دلیل آن بیت اعتبار است که هنوز برای هیچ خانه‌ای یک نشده؛ همان سازوکاری که در بند ۲.۹ توضیح داده شد. این نخستین و ساده‌ترین آزمون طرح است و نبود آن، همهٔ نتایج بعدی را بی‌معنا می‌کرد.

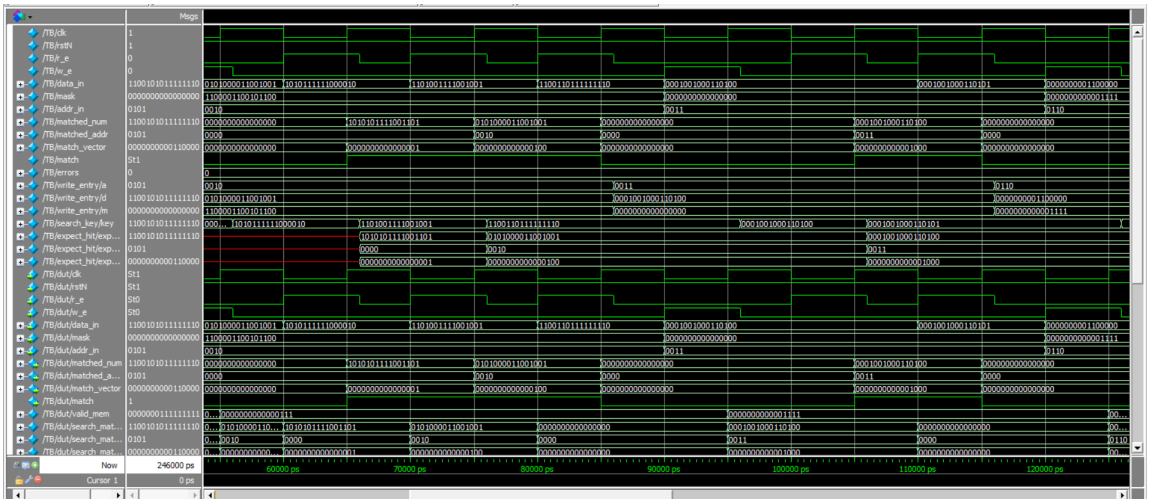
پس از سه نوشتن، دو جست‌وجوی موفّق و یک ناموفّق انجام می‌شود. جدول زیر نشان می‌دهد چرا هر یک نتیجهٔ خود را می‌دهد.

کلید	خانه	بیت‌های معنادار	چرا
$AFC2$	0	$A_C_$	کلید و داده در جایگاه‌های بی‌نقاب یکسان‌اند
$D3C9$	2	نقاب $C32C$	اختلاف‌ها همگی در جایگاه‌های نقاب‌شده‌اند
$CDFE$	—	—	با هیچ خانه‌ای در جایگاه‌های معنادار نمی‌خواند

جدول ۵: سه جست‌وجوی بازه‌ی نخست و دلیل نتیجه‌ی هر یک

نکتهٔ خواندنی این بازه آن است که خانهٔ 1 با نقاب $FF00$ نیز وجود دارد و بایت پایین آن باید برابر EF باشد؛ کلید $CDFE$ بایت پایین FE دارد و با آن نمی‌خواند، پس این جست‌وجو به‌درستی بدون انطباق می‌ماند. اگر بایت پایین کلید EF بود، بایت بالای آن بی‌اهمّیّت می‌شد و انطباق رخ می‌داد.

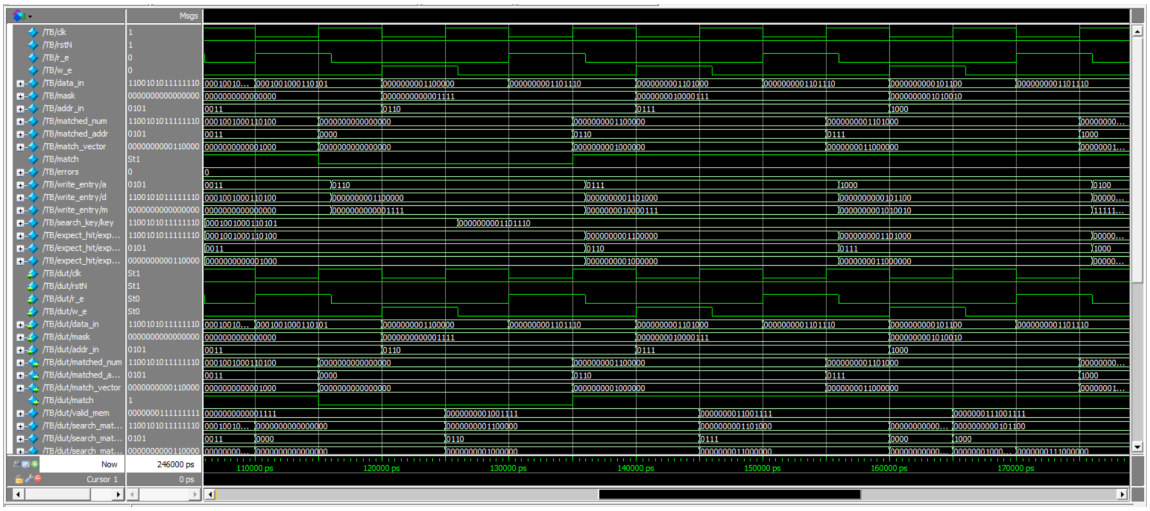
د) بازه‌ی دوم: انطباق دقیق



شکل ۶: نمای بزرگ‌شده‌ی بازه‌ی انطباق دقیق

در لحظهٔ ۹۵ نانوثانیه خانهٔ 3 با داده 1234 و نقاب 0000 نوشته می‌شود. نقاب صفر یعنی هیچ جایگاهی بی‌اهمّیّت نیست و خانه در حالت حافظهٔ شرکت‌پذیر *عادی* کار می‌کند. دو جست‌وجوی بعدی همین را می‌آزمایند: کلید 1234 در لحظهٔ ۱۰۵ منطبق می‌شود و کلید 1235 در لحظهٔ ۱۱۵ منطبق نمی‌شود، هرچند تنها در یک بیت با آن فرق دارد. این جفت وارسی نشان می‌دهد که رفتار سه‌گانه، حالت خاصّ حافظهٔ شرکت‌پذیر متعارف را نیز در بر می‌گیرد.

ه) بازه‌ی سوم: مثال خودِ دستورکار



شکل ۷: نمای بزرگ‌شده‌ی بازه‌ی مثال دستورکار و انباشت سه انطباق

این بازه مهم‌ترین بخش شبیه‌سازی است، چون دقیقاً همان مثالی را بازسازی می‌کند که دستورکار آورده بود. سه الگوی خواسته‌شده به‌ترتیب در خانه‌های 6، 7 و 8 نوشته می‌شوند و پس از هر نوشتن، همان کلید 01101110 جست‌وجو می‌شود. جدول زیر کدگذاری هر الگو و بردار انطباق حاصل را می‌آورد.

زمان (ns)	خانه	الگو	داده	نقاب	بردار انطباق
135	6	0110XXXX	01100000	00001111	خانهٔ 6
155	7	X1101XXX	01101000	10000111	خانه‌های 6, 7
175	8	0X1X11X0	00101100	01010010	خانه‌های 6, 7, 8

جدول ۶: بازسازی مثال دستورکار؛ سه الگوی Xدار در برابر کلید 01101110

درستی هر سه انطباق را می‌توان دستی وارسی کرد. برای الگوی دوم:

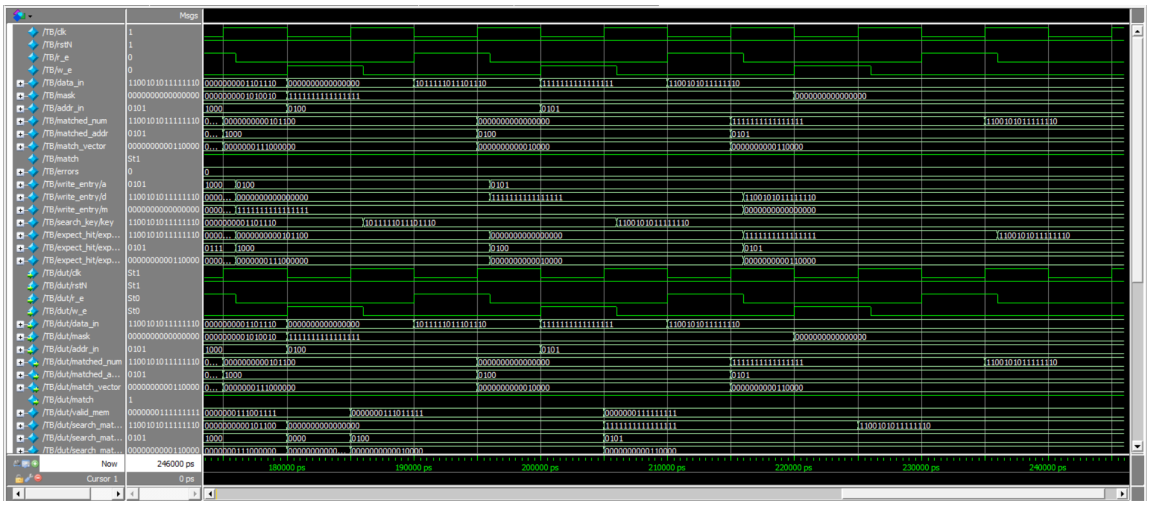
$$01101110 \oplus 01101000 = 00000110, \qquad 00000110 \wedge \overline{10000111} = 00000110 \wedge 01111000 = 0,$$

و برای الگوی سوم:

$$01101110 \oplus 00101100 = 01000010, \qquad 01000010 \wedge \overline{01010010} = 01000010 \wedge 10101101 = 0.$$

در هر دو حالت، همهٔ اختلاف‌ها در جایگاه‌های نقاب‌شده قرار گرفته‌اند و انطباق برقرار است.
آنچه در شکل موج بیش از همه دیدنی است، انباشته شدن بردار انطباق در سه جست‌وجوی پیاپی است: نخست تنها بیت ششم، سپس بیت‌های ششم و هفتم، و سرانجام هر سه بیت ششم تا هشتم بالا می‌روند. این همان چیزی است که دستورکار می‌گوید: هر سه الگو با آن کلید منطبق‌اند. هم‌زمان، matched_addr در سه لحظه به‌ترتیب مقادیر 6، 7 و 8 را می‌گیرد، یعنی همواره بزرگ‌ترین آدرس منطبق را گزارش می‌کند.

و) بازه‌ی چهارم: انطباق فراگیر و بازنویسی



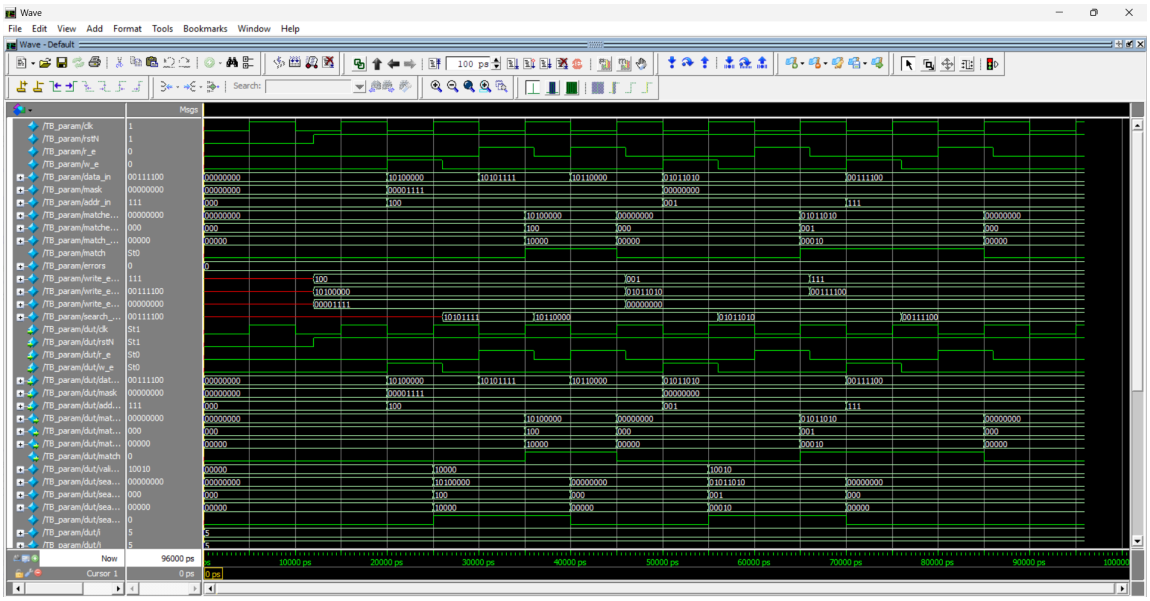
شکل ۸: نمای بزرگ‌شده‌ی بازه‌ی پایانی؛ خانه‌ی سراسر X، انطباق چندگانه و بازنویسی

در لحظهٔ ۱۸۵ نانوثانیه خانهٔ 4 با نقاب $FFFF$ نوشته می‌شود، یعنی خانه‌ای که همهٔ شانزده جایگاه آن بی‌اهمّیّت است. چنین خانه‌ای با هر کلیدی منطبق می‌شود و جست‌وجوی کلید دلخواه $BEEE$ در لحظهٔ ۱۹۵ همین را تأیید می‌کند. این حالت مرزی، هم رفتار درست نقاب کامل را می‌سنجد و هم یادآور می‌شود که چرا بیت اعتبار لازم است: پیش از نوشتن، همین خانه نباید با هیچ کلیدی منطبق می‌شد.

در لحظهٔ ۲۰۵ خانهٔ 5 نیز با نقاب سراسر یک نوشته می‌شود. اکنون دو خانهٔ فراگیر وجود دارد و جست‌وجوی کلید $CAFE$ در لحظهٔ ۲۱۵ هر دو را روشن می‌کند؛ بردار انطباق مقدار 0030_{16} می‌گیرد که بیت‌های چهارم و پنجم آن یک است، و matched_addr مقدار 5 را نشان می‌دهد، یعنی بزرگ‌ترین آدرس.

سرانجام در لحظهٔ ۲۲۵، خانهٔ 5 با داده $CAFE$ و نقاب صفر بازنویسی می‌شود. جست‌وجوی بعدی در لحظهٔ ۲۳۵ نکتهٔ ظریفی را آشکار می‌کند: بردار انطباق همچنان همان 0030_{16} است، ولی به دلیلی متفاوت. پیش از بازنویسی، خانهٔ 5 به‌سبب نقاب فراگیر با هر کلیدی منطبق می‌شد؛ پس از بازنویسی، تنها به این دلیل منطبق است که محتوای آن دقیقاً برابر همان کلید $CAFE$ است. خروجی matched_num این تفاوت را آشکار می‌کند و از $FFFF$ به $CAFE$ تغییر می‌یابد. این وارسی نشان می‌دهد که بازنویسی واقعاً محتوای پیشین را جایگزین کرده است و نه اینکه خانهٔ تازه‌ای ساخته باشد.

ز) پیکربندی پارامتری



شکل ۹: شکل موج اجرای میز آزمون پارامتری با پهنای ۸ بیت و عمق ۵ خانه

در این اجرا، همان پیمانه با پهنای داده هشت بیت و عمق پنج خانه ساخته شده است. پهنای آدرس به‌طور خودکار سه بیت محاسبه می‌شود و بردار انطباق نیز به پنج بیت کوتاه می‌شود؛ هر دو در شکل موج قابل مشاهده‌اند. سه وارسی نخست همان رفتارهای نقاب‌دار، ناموفّق و دقیق را در مقیاس کوچک‌تر تکرار می‌کنند.

وارسی چهارم مخصوص همین پیکربندی است. آدرس 7 روی گذرگاه سه‌بیتی گذاشته می‌شود، حال آنکه حافظه تنها پنج خانه با آدرس‌های صفر تا چهار دارد. در شکل موج دیده می‌شود که با وجود فعّال بودن اجازهٔ نوشتن، هیچ بیت اعتباری تغییر نمی‌کند و جست‌وجوی بعدی نیز بدون انطباق می‌ماند. این همان وارسی $\text{addr_in} < \text{DEPTH}$ است که در بند ۳.۹ توضیح داده شد و اهمّیّت آن تنها در عمق‌های غیرتوان دو آشکار می‌شود.

ح) جمع‌بندی تحلیل

- هر رویداد شبیه‌سازی دقیقاً یک پالس ساعت طول می‌کشد و کلّ اجرا زنجیره‌ای منظّم از لحظه‌های ۲۵ تا ۲۳۵ نانوثانیه است.
- بیت اعتبار در نخستین جست‌وجو، پیش از هر نوشتنی، از انطباق کاذب جلوگیری کرد.
- مثال خودِ دستورکار با هر سه الگوی Xدار بازسازی شد و بردار انطباق در سه گام پیاپی هر سه خانه را روشن کرد.
- در همهٔ حالت‌های انطباق چندگانه، خروجی تک‌مقداری بزرگ‌ترین آدرس را داد و بردار، همهٔ انطباق‌ها را نگه داشت.
- بازنویسی خانه، محتوای پیشین را واقعاً جایگزین کرد؛ تفاوت آن در تغییر matched_num از $FFFF$ به $CAFE$ دیده شد.
- پیکربندی پارامتری بدون هیچ تغییری در کد اجرا شد و وارسی آدرس خارج از محدوده نیز درست عمل کرد.

۷.۹ سنتز و تحلیل زمانی

الف) مصرف منابع

طرح در Quartus II 13.0.1 و برای خانوادهٔ Cyclone IV GX همگردانی شد. انتخاب قطعه روی حالت خودکار بود و ابزار قطعهٔ EP4CGX22CF19C6 را برگزید. نتیجه ۶۵۴ عنصر منطقی شامل ۵۴۶ تابع ترکیبی و ۵۶۵ ثبات اختصاصی، و ۷۷ پایه است.

Flow Summary	
Flow Status	Successful - Wed Aug 26 11:34:38 2026
Quartus II 64-Bit Version	13.0.1 Build 232 06/12/2013 SP 1 SJ Web Edition
Revision Name	TCAM
Top-level Entity Name	TCAM
Family	Cyclone IV GX
Total logic elements	654 / 21,280 (3 %)
Total combinational functions	546 / 21,280 (3 %)
Dedicated logic registers	565 / 21,280 (3 %)
Total registers	565
Total pins	77 / 167 (46 %)
Total virtual pins	0
Total memory bits	0 / 774,144 (0 %)
Embedded Multiplier 9-bit elements	0 / 80 (0 %)
Total GXB Receiver Channel PCS	0 / 4 (0 %)
Total GXB Receiver Channel PMA	0 / 4 (0 %)
Total GXB Transmitter Channel PCS	0 / 4 (0 %)
Total GXB Transmitter Channel PMA	0 / 4 (0 %)
Total PLLs	0 / 4 (0 %)
Device	EP4CGX22CF19C6
Timing Models	Final

شکل ۱۰: خلاصه‌ی منابع مصرفی پیمانه‌ی TCAM

هر دو عدد کلیدی این گزارش را می‌توان مستقیماً از روی کد بازسازی کرد. شمار ثبات‌ها برابر است با آرایهٔ داده، آرایهٔ نقاب، بیت‌های اعتبار و چهار ثبات خروجی:

$$\underbrace{16 \times 16}_{\text{داده}} + \underbrace{16 \times 16}_{\text{نقاب}} + \underbrace{16}_{\text{اعتبار}} + \underbrace{16 + 4 + 16 + 1}_{\text{خروجی‌ها}} = 565,$$

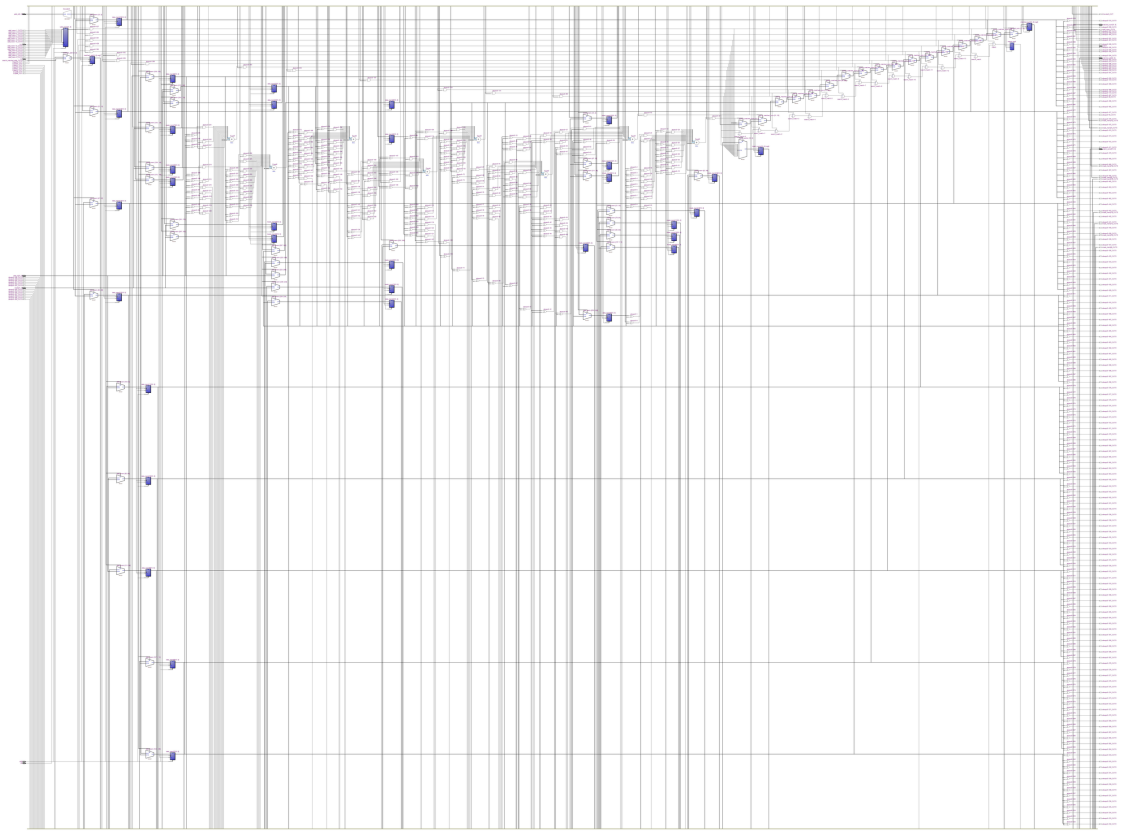
که دقیقاً با عدد گزارش‌شده می‌خواند. شمار پایه‌ها نیز برابر است با چهل پایهٔ ورودی و سی و هفت پایهٔ خروجی:

$$\underbrace{1 + 1 + 1 + 1 + 16 + 16 + 4}_{40} + \underbrace{16 + 4 + 16 + 1}_{37} = 77.$$

نکتهٔ درخور توجّه آن است که مقدار بیت‌های حافظهٔ مصرف‌شده صفر گزارش شده است. یعنی سنتزکننده هیچ بلوک حافظهٔ آماده‌ای را به کار نگرفته و کلّ آرایهٔ ۵۱۲ بیتی را با ثبات‌های معمولی ساخته است. این نتیجه اجتناب‌ناپذیر است و ماهیّت TCAM را نشان می‌دهد: بلوک‌های حافظهٔ آماده تنها یک خانه را در هر دسترسی می‌خوانند، حال آنکه اینجا باید محتوای همهٔ خانه‌ها هم‌زمان و به‌صورت موازی با کلید مقایسه شود. همین نیاز به دسترسی هم‌زمان است که TCAM را نسبت به حافظهٔ معمولی پرهزینه می‌کند.

ب) نماهای ساختاری

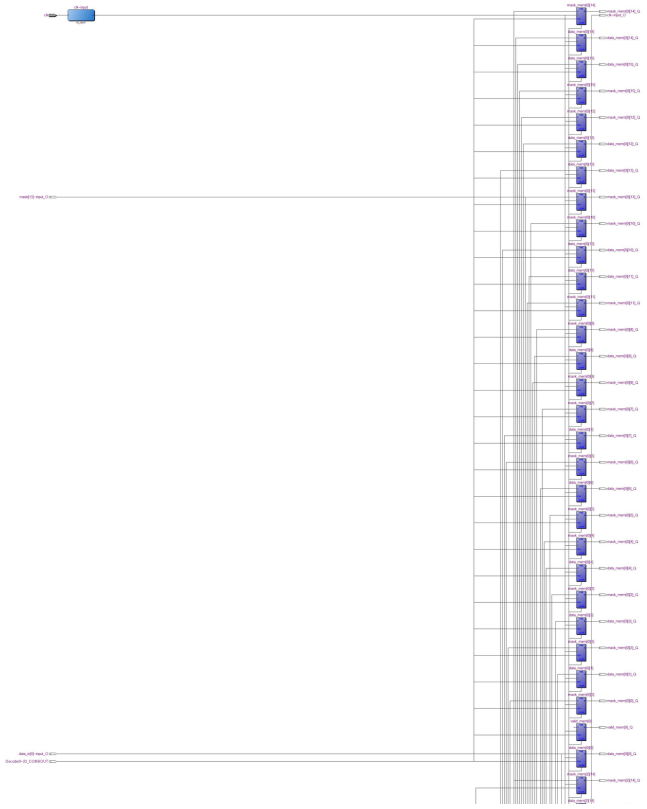
نمای سطح انتقال ثبات‌ها[10] شانزده مسیر مقایسهٔ موازی را در کنار هم نشان می‌دهد؛ هر مسیر شامل یک یای‌انحصاری شانزده‌بیتی، یک واو با نقیض نقاب و یک آشکارساز صفر است. خروجی این مسیرها به منطق اولویّت و ثبات‌های خروجی می‌رسد.



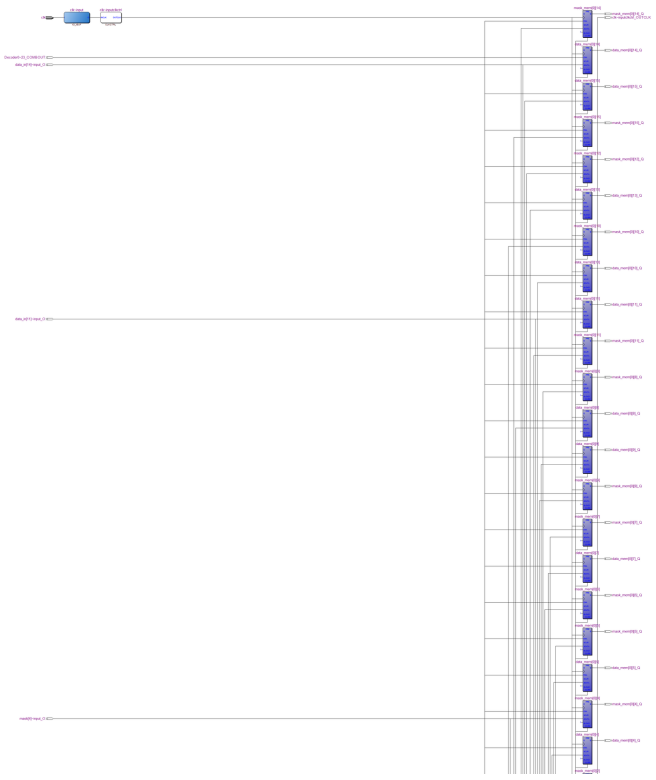
شکل ۱۱: نمای سطح انتقال ثبات‌های پیمانه‌ی TCAM

نماهای فناوری[11] پس از نگاشت و پس از جانمایی نیز ثبت شده‌اند. این دو نما به‌سبب موازی بودن شانزده مسیر مقایسه بسیار بلندند و در عرض یک صفحه خوانا نمی‌شوند؛ ازاین‌رو تنها بخش بالایی هر یک در زیر آورده شده و نسخهٔ کامل در بستهٔ تحویلی است.

[10] Register Transfer Level (RTL) View
[11] Technology Map Viewer



شکل ۱۲: بخشی از نمای Technology Map Viewer (Post-Mapping)



شکل ۱۳: بخشی از نمای Technology Map Viewer (Post-Fitting)

ج) تحلیل زمانی

برای این طرح یک پروندهٔ قیود طراحی[۱۲] نوشته شد که ساعتی با دورهٔ ۱۰ نانوثانیه، معادل ۱۰۰ مگاهرتز، تعریف می‌کند، عدم‌قطعیّت ساعت را استخراج می‌نماید و مسیر ورودی بازنشانی را از تحلیل هم‌زمان کنار می‌گذارد. این کنارگذاری درست است، چون بازنشانی ناهم‌زمان است و نباید مانند دادهٔ هم‌زمان نسبت به ساعت سنجیده شود.

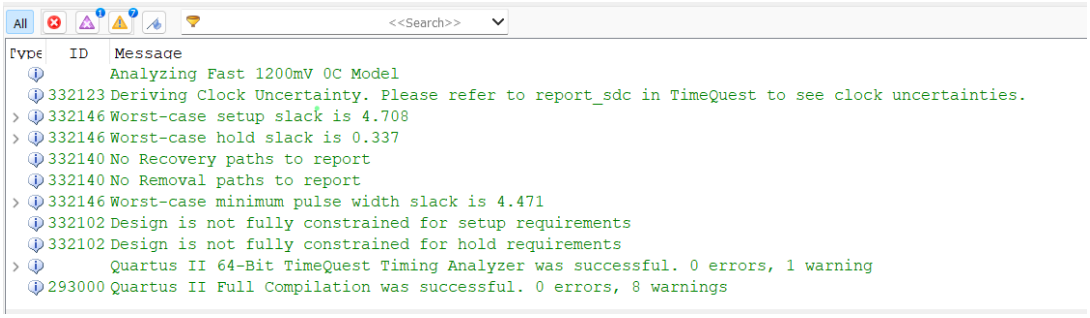
```
# ============================================================
# Experiment 9 - TCAM Timing Constraints
# Functional simulation clock: 100 MHz (10 ns period)
# ============================================================

# Main clock: 100 MHz
create_clock -name clk -period 10.000 -waveform {0.000 5.000} [get_ports {clk}]

# Let TimeQuest calculate device-specific clock uncertainty
derive_clock_uncertainty

# rstN is an asynchronous active-low reset, so it is not timed as
# synchronous data relative to clk.
set_false_path -from [get_ports {rstN}]
```

کد ۶: پرونده‌ی قیود طراحی



شکل ۱۴: پیام‌های پایانی همگردانی موفّق

تحلیل‌گر زمانی[۱۳] در بدترین شرایط کاری حاشیهٔ برپایی[۱۴] 0.758 نانوثانیه و حاشیهٔ نگهداری 0.646 نانوثانیه گزارش می‌کند و مجموع حاشیه‌های منفی در همهٔ تحلیل‌ها صفر است. بیشینه‌بسامد[۱۵] نیز 108.2 MHz است و بیشینه‌بسامد مقیّد همین مقدار است؛ یعنی محدودکنندهٔ سرعت، منطق درونی طرح است و نه نرخ تغییر پایه‌ها.

[12] Synopsys Design Constraints
[13] TimeQuest Timing Analyzer
[14] Slack
[15] Maximum Frequency (Fmax)

Fmax Summary

	Fmax	Restricted Fmax	Clock Name	Note
1	108.2 MHz	108.2 MHz	clk	

شکل ۱۵: گزارش بیشینه‌بسامد پیمانه‌ی TCAM

این حاشیه از همهٔ آزمایش‌های پیشین تنگ‌تر است و دلیل آن در ماهیّت طرح نهفته است. مسیر بحرانی از ورودی کلید آغاز می‌شود، از یای‌انحصاری و نقاب شانزده خانه می‌گذرد، سپس باید نتیجهٔ هر خانه در یک درخت آشکارساز صفر شانزده‌بیتی کاهش یابد و سرانجام از منطق اولویّت انتخاب بزرگ‌ترین آدرس عبور کند. این زنجیرهٔ ترکیبی طولانی، بهای همان جست‌وجوی موازی است که مزیّت اصلی TCAM به شمار می‌رود. بااین‌حال قید ۱۰۰ مگاهرتزی برآورده شده و طرح با حاشیهٔ مثبت بسته می‌شود.
با توجّه به برگزاری مجازی آزمایشگاه، ارزیابی این آزمایش بر پایهٔ شبیه‌سازی، سنتز و تحلیل زمانی نرم‌افزاری انجام شده و در پرونده‌های پیکربندی پروژه تخصیص پایه‌ای برای برد ثبت نشده است؛ ازاین‌رو ادّعایی دربارهٔ آزمون فیزیکی روی برد مطرح نمی‌شود.

۸.۹ نتایج

- پیمانهٔ TCAM با همان اندازهٔ خواسته‌شدهٔ دستورکار، یعنی ۱۶ خانهٔ ۱۶ بیتی، پیاده‌سازی شد.
- مقدار سه‌گانه با زوج داده و نقاب کد شد و شرط انطباق به یک رابطهٔ بیتی ساده تبدیل گردید.
- مثال خودِ دستورکار با هر سه الگوی Xدار بازسازی و در شبیه‌سازی تأیید شد؛ هر سه با کلید 01101110 منطبق شدند.
- بیت اعتبار از انطباق کاذب خانه‌های نانوشته جلوگیری کرد و این رفتار جداگانه آزموده شد.
- جست‌وجو موازی است و زمان آن مستقل از شمار خانه‌هاست؛ در همهٔ حالت‌ها نتیجه در یک پالس ساعت آماده شد.
- در انطباق چندگانه، خروجی تک‌مقداری بزرگ‌ترین آدرس را داد و بردار انطباق همهٔ خانه‌های منطبق را گزارش کرد.
- طرح با پیکربندی هشت‌بیتی و عمق پنج نیز بدون تغییر در کد اجرا شد و وارسی آدرس خارج از محدوده درست عمل کرد.

- سنتز با ۶۵۴ عنصر منطقی، ۵۶۵ ثبات و ۷۷ پایه انجام شد؛ هر دو عدد ثبات و پایه دقیقاً با محاسبهٔ نظری از روی کد می‌خوانند.
- تحلیل زمانی با قید ۱۰۰ مگاهرتزی حاشیهٔ مثبت 0.758 نانوثانیه و بیشینه‌بسامد 108.2 MHz داد.

نتیجه‌گیری

در این آزمایش یک حافظهٔ شرکت‌پذیر سه‌گانه ساخته شد که به‌جای آدرس، با محتوا پرس‌وجو می‌شود. کلید طراحی، تبدیل مفهوم انتزاعی «بیت بی‌اهمّیّت» به یک زوج داده و نقاب بود؛ تبدیلی که شرط انطباق را به یک رابطهٔ بیتی سه‌عملگری فرو کاست و کلّ جست‌وجو را به منطق ترکیبی ساده تبدیل کرد. در کنار آن، دو جزئیّات کوچک اما تعیین‌کننده بودند: بیت اعتبار که بدون آن خانه‌های نانوشته انطباق کاذب می‌ساختند، و تفکیک خروجی تک‌مقداری از بردار انطباق که بدون آن اطّلاعات انطباق‌های هم‌زمان از دست می‌رفت. سرانجام، گزارش سنتز یادآور شد که مزیّت این حافظه رایگان نیست: صفر بودن بیت‌های حافظهٔ مصرف‌شده نشان داد که کلّ آرایه ناگزیر با ثبات ساخته می‌شود، و تنگ بودن حاشیهٔ زمانی نشان داد که جست‌وجوی موازی، مسیر ترکیبی بلندی می‌سازد.